

Session 9 — Modèles économiques et open source en entreprise

Objectifs de la séance

- Identifier les principaux modèles économiques de l'open source.
- Analyser les tensions entre communauté et *business*.
- Comprendre le rôle d'un **OSPO** et de l'**InnerSource**.
- Évaluer les obligations de conformité d'une entreprise.

Partie 1 — Le paradoxe de l'open source

Comment gagner de l'argent avec du « gratuit » ?

■ « *If the software is free, how do you make money?* »

Réponse courte : le logiciel est libre, pas les services associés.

Réponse longue : plusieurs modèles économiques viables existent, chacun avec ses compromis.

Ce que « gratuit » signifie vraiment

Ce qui est gratuit : le code source ; le droit d'utiliser, de modifier, de redistribuer.

Ce qui ne l'est pas forcément : le support, l'expertise, l'intégration dans un SI, les garanties, les fonctionnalités premium d'un éditeur.

L'économie de l'abondance

Dans le monde physique, rareté → valeur. Dans le monde logiciel, **abondance** : la copie est gratuite, la distribution quasi gratuite. Où est la valeur ?

- **Temps et expertise** (toujours rares).
- **Confiance et réputation**.
- **Personnalisation et intégration**.
- **Support et garanties**.

Le coût marginal du logiciel est nul

- **Coût de création** : élevé (développement, tests, documentation).
- **Coût de reproduction** : **zéro** (copier un fichier).
- **Coût de distribution** : quasi nul (Internet).

Conséquences fondamentales :

- Le logiciel est un bien **non-rival** : mon usage ne diminue pas le vôtre.
- Vendre des copies n'a de sens que grâce au **droit d'auteur** (exclusivité artificielle).
- L'open source **supprime** cette exclusivité : il faut vendre **autre chose** que la copie.
- D'où les modèles fondés sur services, support, confiance, expertise.

■ C'est cette propriété économique qui rend l'open source possible — et qui oblige à repenser les modèles de revenus.

Partie 2 — Les modèles économiques

Vue d'ensemble

Modèle	Description	Exemples
Dual licensing	GPL + licence commerciale	MySQL, Qt
Open core	Cœur libre + plugins payants	GitLab, Rudder, Centreon
Distributeur	Intégration + support	Red Hat, SUSE

Modèle	Description	Exemples
Services	Consulting, formation, intégration	Smile, Oslandia
SaaS	Version hébergée	GitLab.com, Clever Cloud
Delayed OSS	Code ouvert après délai (BSL...)	MariaDB, HashiCorp
Donations / sponsors	Financement communautaire	curl, Vue.js

Dual licensing

Même code, **deux licences** :

- **Licence copyleft** (GPL) — gratuit, copyleft s'applique, pour les projets libres.
- **Licence commerciale** — payante, pas de copyleft, pour l'usage propriétaire.

Exemples : MySQL, Qt. Nécessite le **contrôle du copyright** (CLA ou cession). Perçu parfois comme « exploitation » car le projet reçoit des contributions gratuites mais seul l'éditeur peut les vendre en propriétaire.

Open core

Cœur libre + extensions propriétaires :

Enterprise Features	← propriétaire, payant
Core Features	← open source, gratuit

Exemples : GitLab (CE vs EE), Grafana, Rudder, Centreon.

Le dilemme : trop de features dans le core → pas de revenus ; trop peu → communauté frustrée.

Distributeur / support : le modèle Red Hat

Vendre du support et de l'intégration autour de logiciels 100 % libres.

Ce qui est gratuit : le code source (Fedora, CentOS Stream) ; les binaires des *rebuilt*s (Rocky Linux, AlmaLinux).

Ce qui est payant : RHEL (binaires officiels de Red Hat), support 24/7, certifications, garanties, responsabilité contractuelle.

Rachat de Red Hat par IBM (2019) : **34 Md \$**.

Services professionnels — transversal à tous les modèles

Quel que soit le modèle principal (open core, dual, SaaS...), les **services à haute valeur ajoutée** sont une source majeure de revenus pour les éditeurs.

Types de services : consulting et architecture, développement sur mesure, formation et certification, intégration et migration, support et SLA.

Pourquoi c'est important : revenus récurrents et prévisibles, fidélisation des clients, remontée de besoins du terrain, et surtout — ça marche même sans licence propriétaire.

Réputation et confiance — « Red Hat, c'est comme Evian »

« Red Hat c'est comme Evian : l'eau est gratuite. Pourtant Evian vend des milliards de bouteilles. Pourquoi ? Parce que vous faites confiance à la marque. » — Bob Young, ancien CEO de Red Hat (1998).

Ce que vend vraiment un éditeur open source :

- **Garantie** : ça marche, et si ça casse, quelqu'un répond.
- **Certification** : matériel, cloud, conformité (FIPS, Common Criteria...).
- **Réputation** : « *personne ne s'est fait virer pour avoir choisi Red Hat* ».
- **Responsabilité juridique** : indemnisation en cas de violation de licence.

La certification crée un **écosystème** (partenaires, compétences, emplois) qui renforce la position de l'éditeur.

SaaS et le Cloud Problem

SaaS : héberger le logiciel et vendre l'accès. Revenus récurrents, contrôle de l'expérience, scalabilité.
Ex : GitLab.com, MongoDB Atlas, Clever Cloud.

Le Cloud Problem : la licence open source autorise **n'importe qui** à proposer le logiciel en service — y compris AWS, GCP, Azure.

Scénario type :

1. Un éditeur crée une BDD open source (Elasticsearch).
2. Il finance le développement, anime la communauté.
3. AWS lance *Amazon OpenSearch Service* — le même logiciel, hébergé par AWS.
4. AWS a plus de clients, plus de data centers, plus de force commerciale.
5. L'éditeur original perd des clients SaaS.

C'est **légal**, mais c'est l'éditeur qui a tout financé. « *We built it, they sell it.* » — refrain partagé par MongoDB, Elastic, Redis...

Les réponses au Cloud Problem

Réponse 1 — Changer de licence pour exclure le cloud public : SSPL (MongoDB) interdit d'offrir le logiciel comme service ; BSL (MariaDB) devient open source après un délai ; Elastic License impose des restrictions cloud. Controverse majeure : ces licences **ne sont pas open source** selon l'OSI.

Réponse 2 — Différenciation sans changement de licence : proposer des features exclusives à sa propre version SaaS (GitLab), offrir une UX supérieure, miser sur l'intégration, le support et la certification, capitaliser sur la marque et la confiance.

Licences « chrono-dégradables » (*Delayed Open Source*)

Code propriétaire ou restreint pendant un temps, puis open source :

Modèle	Mécanisme	Exemple
Ghostscript (1990s)	Version N-1 libérée quand la N sort	Précurseur historique
BSL	Conversion automatique après un délai fixe (typiquement 3-4 ans)	MariaDB, HashiCorp
Sponsorware	Features libérées quand un seuil de sponsoring est atteint	Material for MkDocs

Material for MkDocs (squidfunk) : les features *Insiders* sont réservées aux sponsors GitHub, libérées sous MIT quand un niveau de financement mensuel est atteint.

Limites : dépendance aux intermédiaires (GitHub Sponsors, Open Collective) qui prélèvent une commission et contrôlent la relation avec les sponsors.

Donations et sponsors

Plateformes de financement communautaire : GitHub Sponsors, Open Collective, Tidelift (orienté entreprise).

Financement public : NLnet (Europe), Sovereign Tech Fund (Allemagne), NGI — Next Generation Internet (UE).

Exemples de réussites : **Vue.js** (Evan You), Babel, Godot Engine.

Limite : rarement suffisant pour vivre à plein temps, sauf pour quelques « superstars ».

L'écosystème open source français

Entreprise	Modèle	Activité
Nexedi	Services + SaaS	ERP5, SlapOS
XWiki	Open core + SaaS	Wiki d'entreprise, CryptPad
Rudder	Open core	Automatisation IT / conformité
Centreon	Open core	Supervision IT
Smile	Services / intégration	Plus grand intégrateur européen
Clever Cloud	SaaS (PaaS)	Hébergement cloud
BlueMind	Open core	Messagerie collaborative
Enalean	Open core + services	Tuleap (ALM)

Structuration : **CNLL** (union des entreprises du libre et du numérique ouvert), **APELL** au niveau européen.

Partie 3 — Cas d'études et tensions

Cas 1 — HashiCorp change de licence (2023)

Août 2023 : Terraform passe de **MPL** → **BSL**. Avril 2024 : **IBM rachète HashiCorp** pour 6,4 Md \$. La communauté réagit en créant **OpenTofu** (Linux Foundation). Le rachat peu après renforce le soupçon de « *bait and switch* » pré-acquisition.

Cas 2 — Redis et Valkey (2024)

Mars 2024 : Redis (la société, ex-Redis Labs) fait passer le **cœur de Redis** (à partir de la version 7.4) de BSD 3-clauses à un double régime **RSALv2** / **SSPLv1** — donc hors open source au sens de l'OSI. Fork **Valkey** aussitôt lancé sous l'égide de la Linux Foundation, soutenu par AWS, Google, Oracle.

Cas 3 — WordPress vs WP Engine (2024)

Matt Mullenweg (BDFL WordPress, CEO Automattic) reproche à WP Engine — hébergeur WordPress majeur — de ne pas contribuer suffisamment.

Arguments de Mullenweg : WP Engine profite massivement de l'écosystème WordPress ; aucun contributeur à la *core team* ; pas de contribution financière à la fondation.

Arguments de WP Engine : ils contribuent via des plugins et l'animation de l'écosystème ; les moyens de pression employés par Automattic sont disproportionnés ; il y a confusion entre les intérêts de la fondation et ceux d'Automattic.

Question soulevée : quand une seule personne contrôle le projet **et** l'entreprise commerciale, qui arbitre ?

La tension communauté / business

Intérêts de la communauté : tout devrait être libre, gouvernance ouverte, pas de « faux » open source (source-available déguisé).

Intérêts business : revenus durables, protection contre les « parasites » (cloud providers notamment), contrôle stratégique de la roadmap.

Équilibre difficile à trouver et à maintenir.

Rappel – comment fonctionne le Venture Capital

Le VC (capital-risque) finance des startups en échange de parts du capital.

1. La startup a une **valorisation** (ex. 10 M€ *pre-money*).
2. Le VC investit (ex. 5 M€) → valorisation *post-money* = 15 M€, VC détient 33 %.
3. Le VC a des **actions de préférence** (liquidation prioritaire, anti-dilution).
4. L'objectif du VC : un **exit** (revente ou IPO) avec un multiple d'au moins $\times 10$ en 5-7 ans.

Conséquence pour l'open source : le VC n'investit pas par philanthropie. Il exige une croissance agressive et un exit lucratif. Si le modèle open source ne génère pas assez de revenus → pression pour changer la licence ou se faire racheter.

Le *bait and switch*

Pattern récurrent chez les éditeurs VC-funded :

1. Lancer un projet open source → adoption rapide, communauté gratuite.
2. Lever des fonds (VC) sur la base de cette adoption.
3. Construire un produit commercial autour du projet.
4. **Changer la licence** une fois établi (SSPL, BSL, Elastic License...).
5. Monétiser la communauté captive – ou se faire racheter (exit pour le VC).

Exemples : MongoDB (IPO), Elastic (IPO), HashiCorp (racheté IBM), Redis, CockroachDB, Sentry...

Signes d'alerte

Projet potentiellement à risque	Projet plus sûr
<i>Single-vendor</i>	Multi-vendor
CLA exigeant (<i>copyright assignment</i>)	Fondation neutre
Financement VC important	DCO simple
Pas de gouvernance communautaire	Gouvernance transparente

VC-funded vs bootstrapped

VC-funded → pression de sortie	Bootstrapped → stabilité
Croissance rapide exigée	Croissance organique
Exit (IPO ou rachat) attendu	Pas de pression actionnariale
Changement de licence fréquent	Licence stable dans le temps
Ex : MongoDB, HashiCorp, Elastic	Ex : XWiki, Nexedi, Enalean

Les **projets communautaires** (PostgreSQL, Linux) évitent la pression capitaliste mais ont d'autres risques : désengagement des contributeurs faute de modèle économique (Apache Attic, Apache OpenOffice après le retrait d'IBM) ou conflits fondation vs éditeur principal (TDF vs Collabora, cf. session 8).

Open source en entreprise

Face à ces enjeux — modèles économiques, tensions communautaires, risques de conformité — les entreprises doivent **s’organiser**. Trois réponses complémentaires :

1. **Conformité** — respecter les obligations légales des licences.
2. **OSPO** — gouverner l’usage et la contribution open source.
3. **InnerSource** — appliquer les pratiques open source en interne.

Partie 4 — Conformité

Obligations selon les licences

■ Rappel des sessions 4-5, appliqué au contexte entreprise.

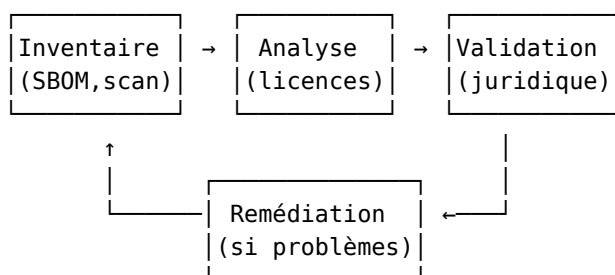
Licences permissives (MIT, BSD, Apache...) : conserver les notices de copyright, inclure le texte de la licence, parfois afficher un disclaimer de garantie.

Licences copyleft (GPL, LGPL, AGPL...) : mêmes obligations que le permissif, plus fournir le code source des modifications et conserver la même licence dans toute redistribution.

Risques de non-conformité

Entreprise	Violation	Conséquence
Cisco (2008)	GPL dans Linksys	Procès FSF, <i>settlement</i>
Free (2011)	GPL Freebox	Condamné (France), code publié
VMware (2015)	Linux kernel	Procès, abandonné
Hancom (2016)	GPL Ghostscript	Condamné (Corée)
SFC vs Vizio (2021)	GPL	<i>Settlement</i> 2025
Orange (2011→2024)	GPL v2 (Lasso)	Condamné à 800 k€ (France)

Processus et outils



Outil	Type	Fonctionnalités
REUSE (FSFE)	Open source	Vérification métadonnées licence/copyright par fichier
FOSSology	Open source	Scan, identification, reporting
ScanCode	Open source	Détection de licences
Black Duck	Commercial	Enterprise, compliance
FOSSA	Commercial	Intégration CI/CD

Partie 5 — L’OSPO (*Open Source Program Office*)

Qu’est-ce qu’un OSPO ?

Structure dédiée à la **gestion de l’open source** en entreprise.

Missions principales : définir la politique d'utilisation, s'assurer de la conformité des licences, coordonner la contribution *upstream*, animer les relations avec les communautés externes.

Rattachement hiérarchique typique : CTO ou direction technique, direction juridique, ou structure transverse indépendante. L'OSPO travaille en transverse, au service de toutes les équipes.

Pourquoi créer un OSPO ?

Enjeux pour l'entreprise :

1. **Juridiques** — conformité, gestion des risques.
2. **Sécurité** — *supply chain*, vulnérabilités (cf. session 10).
3. **Stratégiques** — influence sur les standards, recrutement, innovation.
4. **Économiques** — optimisation des coûts, éviter le *vendor lock-in*.

Les entreprises utilisent en moyenne **500+** composants open source.

Responsabilités

Côté consommation d'open source : inventaire des dépendances, validation des licences, gestion des vulnérabilités, formation des développeurs.

Côté contribution : définir une politique de contribution interne, mettre en place un processus de validation, entretenir les relations avec les projets amont, financer (sponsoring, donations).

Exemples

Entreprise	OSPO depuis	Focus
Google	2004	Innovation, Android, Kubernetes
Microsoft	2014	Transformation, GitHub
Red Hat	Origine	Cœur de métier
SAP	2017	Conformité, contribution
Spotify	2018	InnerSource, outils
SNCF	2020	Souveraineté

Communauté des OSPO : **TODO Group** (Linux Foundation) — <https://todogroup.org/>.

Partie 6 — InnerSource

Qu'est-ce que l'InnerSource ?

Appliquer les pratiques open source à l'intérieur de l'entreprise.

Open Source	InnerSource
Code public	Code interne partagé
Contributeurs externes	Contributeurs = employés
Communauté mondiale	Communauté entreprise
Gouvernance ouverte	Gouvernance adaptée

Problèmes résolus

Situation typique :

- Équipe A développe un composant.
- Équipe B a besoin d'une modification.
- Équipe A est occupée → ticket dans le backlog.

- Équipe B attend... ou duplique le code.

Avec InnerSource : équipe B propose une PR, équipe A review et merge. Tout le monde gagne du temps.

Rôles et mise en place

Rôle	Responsabilités
Guest	Contributeur externe à l'équipe
Trusted Committer	Mainteneur, <i>review</i> , mentorat
Product Owner	Vision, priorités, <i>roadmap</i>
Contributor	Développeur qui soumet des PRs

Étapes recommandées :

1. **Pilote** — commencer avec 2-3 projets volontaires.
2. **Outillage** — forge interne (GitLab, GitHub Enterprise).
3. **Culture** — valoriser les contributions *cross-équipes*.
4. **Métriques** — mesurer l'adoption et les bénéfices.

Ressources : **InnerSource Commons** — <https://innersourcecommons.org/>.

Ce qu'il faut retenir

1. **Coût marginal nul** → l'open source oblige à vendre autre chose que la copie.
2. **Plusieurs modèles** viables : services, open core, dual licensing, SaaS, delayed OSS, sponsors.
3. **Réputation et certification** : la confiance est le vrai produit (Bob Young, Red Hat).
4. **Cloud Problem** et **VC-funded** expliquent la plupart des *relicensing* récents.
5. **Conformité** : obligations réelles, risques juridiques, outils disponibles.
6. **OSPO** : structure clé pour gouverner l'open source en entreprise.
7. **InnerSource** : appliquer les pratiques open source en interne.

Pour aller plus loin

- *Open Source Business Models* (OSI) — <https://opensource.org>
- CNLL — <https://cnll.fr>
- TODO Group (OSPO) — <https://todogroup.org>
- InnerSource Commons — <https://innersourcecommons.org>